

100 AWS DevOps Interview Questions & Answers

A complete reference guide for Senior DevOps / AWS Engineer interviews (4-8+ years)

100

QUESTIONS ANSWERED

10

TOPIC AREAS

10

DOMAINS COVERED

Table of Contents

AWS Fundamentals	Questions 1-10
EC2 & Networking	Questions 11-20
Load Balancer & Auto Scaling	Questions 21-30
Storage	Questions 31-40
IAM & Security	Questions 41-50
Docker	Questions 51-60
Kubernetes	Questions 61-70
CI/CD	Questions 71-80
Terraform	Questions 81-90
Linux	Questions 91-100

1. Explain the AWS Global Infrastructure.

AWS infrastructure is built from **Regions** (separate geographic areas, each fully isolated), **Availability Zones** (one or more discrete data centers per region with independent power, cooling, and networking), and **Edge Locations** (CloudFront / Route 53 endpoints that cache content and resolve DNS closer to end users). There are also **Local Zones** and **Wavelength Zones** for ultra-low-latency compute closer to specific cities or 5G networks. As of 2026 AWS spans 30+ regions and 100+ AZs worldwide.

2. Difference between Region, Availability Zone, and Edge Location.

Concept	What it is	Purpose
Region	A geographic area (e.g. ap-south-1, Mumbai) containing multiple AZs	Isolates failures and data; pick the region closest to your users / compliance needs
Availability Zone	One or more physical data centers within a region, independently powered/networked	Spread resources for high availability inside a region
Edge Location	Smaller CloudFront/Route 53 site, no EC2/RDS	Cache content and resolve DNS closer to end users for lower latency

3. How does an Internet Gateway work?

An Internet Gateway (IGW) is a horizontally scaled, redundant, highly available VPC component that lets instances with public IPs communicate with the internet. It performs 1:1 NAT, translating an instance's private IP to its public IP for outbound traffic and back again for the response. It's referenced as a target in a route table (typically `0.0.0.0/0 -> igw-id`) for the public subnet. It does not perform NAT for instances that only have private IPs — that's the job of a NAT Gateway.

4. Explain the purpose of a NAT Gateway.

A NAT Gateway lets instances in a **private** subnet initiate outbound connections to the internet (e.g. to download OS patches or hit an external API) while blocking any inbound connection initiated from the internet. It lives in a public subnet, needs an Elastic IP, and is fully AWS-managed and highly available within its AZ — for multi-AZ resilience you deploy one NAT Gateway per AZ rather than sharing a single one across AZs.

5. Difference between Security Groups and Network ACLs.

	Security Group	Network ACL
Level	Instance / ENI level	Subnet level
State	Stateful — return traffic auto-allowed	Stateless — inbound and outbound rules both needed
Rule types	Allow rules only	Allow and explicit Deny rules
Evaluation	All matching rules are combined (union)	Rules processed in order by rule number, first match wins

6. Explain VPC Peering and its limitations.

VPC Peering creates a private network connection between two VPCs, routing traffic over the AWS backbone instead of the public internet. Key limitations: **no transitive peering** (if A-B and B-C are peered, A still cannot reach C through B), **no overlapping CIDR blocks** are allowed between the peered VPCs, it doesn't support edge-to-edge routing through a gateway or VPN sitting in the peer VPC, and route tables / security groups must be updated manually on both sides.

7. What is AWS Transit Gateway?

Transit Gateway is a regional network hub that lets you connect many VPCs, VPN connections, and Direct Connect gateways through a single gateway instead of building a full mesh of individual peering connections. Unlike VPC peering, it supports **transitive routing** through route tables that control which attachments can talk to each other, and Transit Gateways can themselves be peered across regions for global connectivity.

8. Explain Route Tables in AWS.

Every subnet in a VPC is associated with exactly one route table, which contains rules (routes) that decide where network traffic from that subnet is sent, matched using the most specific destination CIDR (longest prefix match). Each VPC starts with a default *main* route table; you typically create custom route tables per subnet — e.g. a public subnet routes `0.0.0.0/0` to an Internet Gateway, while a private subnet routes it to a NAT Gateway.

9. How does DNS resolution work inside a VPC?

AWS places a VPC DNS resolver at the base of the VPC's CIDR plus two (e.g. `10.0.0.2`), which forwards queries to the Route 53 Resolver to handle both public internet DNS and private hosted zone resolution. The VPC attributes `enableDnsSupport` and `enableDnsHostnames` control whether this resolver is active and whether instances receive DNS hostnames. Route 53 Resolver also supports hybrid DNS via inbound/outbound endpoints for on-prem integration.

10. Explain AWS PrivateLink.

PrivateLink lets you privately access AWS services, your own services, or third-party SaaS exposed as a VPC endpoint service, using an ENI placed directly in your subnet — traffic never crosses the public internet. It avoids the need for VPC peering, an Internet Gateway, or NAT, and is the standard way to securely expose a service to other VPCs/accounts, or consume a SaaS partner's API privately.

11. Explain the complete EC2 boot process.

When you launch an instance, AWS picks a physical host, attaches the boot (root) EBS volume built from your AMI, and allocates an ENI with a private IP (plus a public IP/EIP if requested). The hypervisor initializes hardware, the kernel boots, and `cloud-init` runs to set the hostname, fetch and execute any **User Data** script, and configure SSH keys. The instance then moves to the *running* state once both the system status check and instance status check pass.

12. How do you troubleshoot an unreachable EC2 instance?

Work top-down: confirm the instance state and status checks are healthy; check Security Group inbound rules allow the port/source; check the subnet's NACL rules; verify the route table has a path to an Internet Gateway (for public access) or the right next hop; confirm the instance has a public IP/EIP if reached from the internet; check the OS-level firewall (iptables/ufw/Windows Firewall); review the system log ("Get System Log") for boot errors; and check the EBS volume isn't full or detached, plus VPC DNS settings if hostname resolution fails.

13. Difference between Instance Store and EBS.

	Instance Store	EBS
Attachment	Physically attached to the host	Network-attached block storage
Persistence	Ephemeral — lost on stop/terminate/hardware failure	Persists independently of the instance lifecycle
Performance	Very high IOPS, no network hop	High but bound by network/volume type
Use case	Cache, scratch space, temp data	Root volumes, databases, anything needing durability
Backups	Not snapshot-able	Snapshot, resize, detach/reattach to other instances

14. Explain Placement Groups.

Placement Groups control how EC2 places instances on underlying hardware. **Cluster** packs instances close together within a single AZ for low-latency, high-throughput networking (HPC workloads). **Spread** places each instance on distinct underlying hardware to minimize correlated failure (max 7 instances per AZ). **Partition** divides instances into logical partitions on separate racks, ideal for rack-aware distributed systems like HDFS, Cassandra, or Kafka.

15. How do Security Groups evaluate rules?

Security Groups only support **Allow** rules — there's no explicit Deny. Every matching rule across all attached Security Groups is combined as a union; if any rule allows the traffic, it's allowed. There's no concept of "most specific rule wins" the way NACLs work. Because SGs are **stateful**, a response to allowed inbound traffic is automatically permitted outbound regardless of the outbound rule set, and vice versa.

16. Explain Elastic IP.

An Elastic IP (EIP) is a static public IPv4 address allocated to your account that you associate with an instance or ENI. Unlike the default public IP (which changes every time you stop/start an instance), an EIP persists, making it useful for IP allowlisting or DNS records that must stay fixed. AWS charges for EIPs that are allocated but not attached to a running resource, to discourage hoarding scarce IPv4 addresses.

17. How does ENI (Elastic Network Interface) work?

An ENI is a virtual network card you can attach to an instance, carrying its own private IPv4/IPv6 addresses, MAC address, Security Groups, and optionally an Elastic IP. You can attach multiple ENIs to a single instance (e.g. for dual-homed setups spanning subnets) or detach an ENI from one instance and reattach it to another, preserving its IP and MAC — a common pattern for fast failover without DNS propagation delays.

18. Explain User Data.

User Data is a script (bash, cloud-init directives, or PowerShell on Windows) you pass at launch time, which `cloud-init` executes automatically on first boot to bootstrap the instance — e.g. installing packages, joining a cluster, or pulling config from S3/SSM. It's limited to 16KB and base64-encoded under the hood. By default it only runs once on first boot; running it again on every boot requires explicit cloud-init directives.

19. How do you resize an EC2 instance?

Stop the instance (its data is safe on EBS), change the instance type via the console, CLI (`modify-instance-attribute`), or Terraform, then start it again. Note the instance typically receives a new public IP unless an Elastic IP is attached, and the target instance type must be supported in the instance's current Availability Zone and any Placement Group it belongs to.

20. What happens if the root EBS volume is deleted?

If "Delete on Termination" is set to true (the default for root volumes) and you terminate the instance, the root volume is deleted automatically and unrecoverable unless a snapshot was taken beforehand. If you manually detach and delete the root volume while the instance is stopped, the instance can no longer boot until a new root volume is attached. Best practice for critical workloads: take regular snapshots and set `DeleteOnTermination=false` on production root volumes.

21. Difference between ALB, NLB, and GWLB.

Type	Layer	Best for
ALB (Application LB)	L7	Content-based routing by host/path/header, HTTP/HTTPS/WebSocket, microservices
NLB (Network LB)	L4	Ultra-low latency, millions of req/sec, static IPs, TCP/UDP, preserving source IP
GWLB (Gateway LB)	L3	Transparently routing traffic through 3rd-party appliances (firewalls/IDS) via GENEVE

22. Explain ALB listener rules.

A listener checks for connection requests on a configured protocol/port. Rules attached to it — each with a priority, conditions (path-pattern, host-header, HTTP header/method, query string), and an action (forward, redirect, fixed-response) — decide how matching requests are routed to specific target groups. Rules are evaluated in priority order, with a default rule catching anything that matches nothing else.

23. What are Target Groups?

A Target Group is a logical grouping of targets (EC2 instances, IPs, Lambda functions, or even other ALBs) that a listener forwards matching requests to, along with its own health check configuration and protocol/port. A single Target Group can be attached to multiple listeners or load balancers, and supports weighted routing — useful for blue/green and canary release patterns.

24. How do ALB health checks work?

ALB periodically sends a request (HTTP GET to a configured path, or a TCP check) to every registered target and marks it healthy or unhealthy based on configured thresholds (success codes, interval, timeout, healthy and unhealthy threshold counts). Unhealthy targets stop receiving new traffic until they pass checks again, letting the ALB automatically route around failing instances without manual intervention.

25. Explain sticky sessions.

Sticky sessions bind a client to a particular target for the life of their session, using either an ALB-generated cookie (**AWSALB**) or an application-generated cookie. They're useful when an app stores session state in memory rather than externally (e.g. Redis). The trade-off is uneven load distribution and a broken session if that specific target becomes unhealthy.

26. Difference between Cross-Zone Load Balancing and normal routing.

With cross-zone load balancing enabled, the load balancer spreads traffic evenly across **all** registered targets in **all** enabled AZs, regardless of which AZ actually received the request. Without it, each LB node only routes to targets within its own AZ, which can cause imbalance if instance counts differ per AZ. ALB has this on by default (can't be disabled); NLB lets you toggle it, and charges for the cross-AZ data transfer when enabled.

27. Explain Auto Scaling lifecycle hooks.

Lifecycle hooks (**Pending:Wait** and **Terminating:Wait**) pause an instance in a wait state for a configurable timeout during launch or termination, giving you a window to run custom actions via SNS/Lambda — e.g. installing/configuring software before the instance goes into service, or draining connections and shipping final logs before it's terminated.

28. What scaling policies are available?

Target Tracking — keeps a metric like CPU% at a target value (simplest, most common). **Step Scaling** — adds/removes capacity in steps sized to the magnitude of the alarm breach. **Simple Scaling** — a single scaling adjustment with a cooldown period between actions. **Predictive Scaling** — uses ML to forecast traffic and pre-emptively scale ahead of expected demand.

29. Difference between Scheduled Scaling and Dynamic Scaling.

Scheduled Scaling sets capacity changes at known times (e.g. scale up every weekday at 8am), suited to predictable patterns like office hours or batch jobs. Dynamic Scaling reacts in real time to actual demand via CloudWatch alarms/metrics (e.g. target tracking on CPU). Most production setups combine both — scheduled scaling sets the predictable baseline, dynamic scaling absorbs unexpected spikes.

30. How do you troubleshoot Auto Scaling failures?

Check the ASG's Activity History for the exact failure reason (insufficient capacity, IAM permission errors, missing AMI, bad launch template); confirm the launch template references a valid AMI/instance type available in the target AZs; check service quotas (vCPU limits); review the health check grace period if instances are terminated before the app finishes starting; verify the subnet has free IP addresses; and check whether scaling cooldowns are currently suppressing further actions.

31. Difference between S3 Standard, IA, Glacier, and Deep Archive.

Class	Access	Use case
Standard	Millisecond, frequent access	Active, frequently-read data — highest storage cost
Standard-IA	Millisecond, but retrieval fee	Infrequently accessed but needs fast access when needed, 30-day min
Glacier Instant/Flexible	ms to minutes/hours	Archival data accessed rarely, cheaper storage
Glacier Deep Archive	~12 hours	Long-term compliance/backup, cheapest tier

32. Explain S3 Versioning.

When enabled on a bucket, every PUT, overwrite, or delete creates a new object version keyed by a version ID instead of replacing existing data; a "delete" simply adds a recoverable *delete marker* rather than removing the underlying data. This protects against accidental overwrites or deletions and pairs well with lifecycle rules to expire old versions, and with MFA Delete for extra protection.

33. What is Object Lock?

Object Lock is a WORM (write-once-read-many) feature on a versioned bucket that prevents object versions from being deleted or overwritten for a fixed retention period, or indefinitely under a legal hold. It has two modes: **Governance** (overridable by users with a special permission) and **Compliance** (cannot be altered or deleted by anyone, including the root user, until the retention period expires) — used to meet regulatory/audit requirements.

34. Explain Lifecycle Policies.

Lifecycle Policies automatically transition objects between storage classes (e.g. Standard → IA after 30 days → Glacier after 90 days) or expire/delete them after a set time, scoped by prefix or tag filters. They automate cost optimization without manual intervention, and can also be used to clean up incomplete multipart uploads that would otherwise accumulate hidden storage charges.

35. Difference between EBS, EFS, and FSx.

EBS — block storage attached to a single EC2 instance at a time (except Multi-Attach io1/io2), AZ-bound. **EFS** — NFS-based file storage that many instances across AZs can mount concurrently, scales automatically; good for shared content and web serving. **FSx** — managed file systems for specific needs: FSx for Windows File Server (SMB, AD integration), FSx for Lustre (HPC/ML high-throughput), and FSx for NetApp ONTAP/OpenZFS for advanced enterprise file features.

36. Explain Multipart Upload.

For large objects (recommended above ~100MB, required above 5GB) S3 lets you split an upload into independent parts that are uploaded in parallel and retried individually on failure, then reassembled by S3 once all parts complete. This improves throughput and resilience to network interruptions; incomplete uploads should be cleaned up with a lifecycle rule to avoid storage charges for orphaned parts.

37. How does S3 consistency work?

S3 provides **strong read-after-write consistency** for all operations — PUTs, overwrites, and deletes — across all regions: a GET issued immediately after a successful PUT will always return the latest data. This replaced the older eventual-consistency model that previously applied to overwrite PUTs and DELETES before December 2020.

38. What is S3 Replication?

Cross-Region Replication (CRR) or Same-Region Replication (SRR) automatically and asynchronously copies objects from a source bucket to a destination bucket (in the same or a different account/region), based on rules, and requires versioning enabled on both buckets. It's used for compliance/geo-redundancy, reducing latency by serving data closer to users, or maintaining a separate copy for security and auditing.

39. Explain EBS snapshots.

EBS snapshots are point-in-time, incremental backups of a volume stored in S3 (managed by AWS, not directly browsable). The first snapshot copies all data; each subsequent snapshot only copies blocks changed since the previous one, but every snapshot still represents a complete point-in-time view when restored. Used for backups, creating new volumes/AMIs, and cross-region or cross-account copies for disaster recovery.

40. How do you encrypt storage in AWS?

EBS — enable encryption at volume creation (KMS-backed); can set an account-level default so all new volumes/snapshots are encrypted automatically and transparently, covering data at rest, in transit between instance and volume, and in snapshots. **S3** — SSE-S3 (AWS-managed keys), SSE-KMS (customer-managed keys with a CloudTrail audit trail), or SSE-C (customer-supplied keys); enforce it bucket-wide via bucket policy. RDS and EFS support similar KMS-backed encryption at rest.

41. Explain IAM Roles vs IAM Users.

A **User** is a long-term identity (a person or application) with permanent credentials — a password and/or access keys — used to sign in directly or make API calls. A **Role** is an identity with temporary credentials, assumed by users, applications, or AWS services like EC2 and Lambda. No long-term credentials are stored for a role, which reduces the risk from leaked secrets and is the recommended way to grant permissions to workloads.

42. Difference between Managed Policies and Inline Policies.

Managed policies are standalone, reusable IAM documents (AWS-managed or customer-managed) that can be attached to many users/groups/roles and are versioned. Inline policies are embedded directly inside a single user/group/role, exist only as long as that identity does, and can't be reused elsewhere — simpler for a strict one-to-one relationship, but harder to audit and maintain consistently at scale.

43. Explain AssumeRole.

AssumeRole is an STS API call that lets a trusted principal (a user, role, or AWS service) obtain temporary security credentials — an access key, secret key, and session token — for a *different* IAM role, based on that role's trust policy. It's the standard mechanism for cross-account access, identity federation, and granting Lambda/EC2/ECS workloads scoped permissions without static credentials.

44. What is STS?

Security Token Service issues temporary, limited-privilege credentials for IAM users or federated identities, via calls like AssumeRole, AssumeRoleWithSAML, AssumeRoleWithWebIdentity, and GetSessionToken. These credentials expire automatically (default 1 hour, configurable up to 12 hours), which dramatically reduces the blast radius compared to long-lived static access keys.

45. How do IAM permissions get evaluated?

AWS evaluates every applicable policy — identity-based, resource-based, permission boundaries, SCPs, and session policies — for a given request. The default is implicit deny. An **explicit Deny** in any policy always wins over any Allow. If no explicit deny exists, an explicit Allow from any applicable policy grants access; if nothing grants it, the request is denied by default.

46. Explain SCPs (Service Control Policies).

SCPs are policies applied at the AWS Organizations level — on the org root, an OU, or a member account — that set the maximum permissions ever available to accounts within scope. They don't grant permissions themselves, only restrict what IAM policies inside those accounts can allow, even for the account's root user. Commonly used as guardrails, e.g. "never leave a specific AWS region" or "never disable CloudTrail."

47. What is AWS Organizations?

A service for centrally managing multiple AWS accounts: grouping them into Organizational Units, applying SCPs and tag policies for governance, consolidating billing, and centrally enabling services like GuardDuty, Config, and CloudTrail across every member account. It forms the backbone of a multi-account landing zone strategy used by most mature AWS organizations.

48. Explain KMS.

Key Management Service is a managed service for creating and controlling cryptographic keys (CMKs) used to encrypt data across AWS services (S3, EBS, RDS, etc.) and in custom applications via the KMS API/SDK. It provides centralized key policy management, automatic or manual key rotation, and a detailed CloudTrail audit log recording every single use of a key.

49. Difference between CMK and AWS-managed keys.

A **Customer Managed Key (CMK)** is created and fully controlled by you — you set the key policy, rotation schedule, and can disable or schedule deletion, plus share it across accounts. An **AWS-managed key** (e.g. `aws/s3`) is automatically created and managed by the AWS service on your behalf — simpler to use, but you have no control over its policy, rotation, or deletion, and it can't be shared across accounts.

50. How do you securely store application secrets?

Use **Secrets Manager** (automatic rotation, fine-grained IAM access, native integration with RDS) or **SSM Parameter Store SecureString** (cheaper, KMS-encrypted, ideal when you don't need rotation) rather than hardcoding secrets in code, AMIs, or plaintext environment variables. Applications should fetch secrets at runtime under an IAM role's permissions, with all access logged via CloudTrail.

51. Explain Docker architecture.

Docker uses a client-server model: the Docker CLI (client) talks to the Docker daemon (`dockerd`) over a REST API/socket. The daemon manages images, containers, networks, and volumes, and delegates to `containerd` and `runc` underneath to actually create and run containers using Linux kernel primitives — namespaces for isolation and cgroups for resource limits. Images are pulled from and pushed to registries such as Docker Hub or Amazon ECR.

52. Difference between Image and Container.

An **image** is a read-only, layered template (filesystem plus metadata) defining what a container will run — it's immutable and stored in a registry. A **container** is a running (or stopped) instance of an image, with its own writable layer on top, isolated process namespace, and runtime state — the same relationship as a class (image) to an object instance (container).

53. Explain Docker Layers.

Each instruction in a Dockerfile (`FROM` , `RUN` , `COPY` , etc.) creates a new read-only layer stacked on top of the previous ones, merged by a union filesystem like overlay2. Layers are cached and shared across images, so unchanged layers don't need to be rebuilt or re-downloaded — which is why ordering Dockerfile instructions from least-to-most frequently changing speeds up build times.

54. What is Docker Overlay2?

Overlay2 is the default storage driver on modern Linux Docker installations, implementing a union filesystem that merges multiple read-only image layers with a single writable "upper" layer per container, presenting a unified view of the filesystem inside the container. It uses copy-on-write semantics — files are only copied into the writable layer when modified — and generally outperforms older drivers like AUFS or devicemapper.

55. Difference between CMD and ENTRYPOINT.

`CMD` provides default arguments/command that can be completely overridden by arguments passed to `docker run` . `ENTRYPOINT` defines the fixed executable that always runs; any `CMD` or run-time arguments are appended to it as arguments (unless overridden with `--entrypoint`). Best practice: use `ENTRYPOINT` for the main binary and `CMD` for its default flags, making the image behave like a configurable executable.

56. Explain Docker Networking.

Default drivers: **bridge** (default, an isolated private network per host with NAT out, used for most standalone containers), **host** (container shares the host's network namespace directly — no isolation, max performance), **none** (no networking at all), **overlay** (multi-host networking for Swarm/cluster setups using VXLAN), and **macvlan** (assigns a real MAC address so the container appears as a physical device on the LAN).

57. Difference between Bind Mount and Volume.

A **bind mount** maps a specific host filesystem path directly into the container — fast and simple for local development, but tightly coupled to the host's directory layout and not portable. A **volume** is fully managed by Docker (stored under `/var/lib/docker/volumes`), portable across hosts with the right driver, easier to back up and migrate, and the recommended approach for persistent container data in production.

58. Explain Multi-stage Docker Builds.

Multi-stage builds use multiple `FROM` statements in one Dockerfile, where each stage can use a different base image, and later stages can selectively copy artifacts from earlier ones with `COPY --from=builder`. This lets you compile/build in a heavyweight stage (with compilers and dev tools) while shipping only the compiled binary in a minimal final runtime image, dramatically shrinking image size and attack surface.

59. How do you reduce Docker image size?

Use minimal base images (alpine, distroless, scratch); apply multi-stage builds to drop build tools from the final image; combine RUN commands and clean package caches in the same layer (`apt-get install && rm -rf /var/lib/apt/lists/*`); use a `.dockerignore` file to avoid copying unnecessary files into the build context; and skip installing debug/diagnostic tools in production images.

60. How do you troubleshoot a failing container?

Check `docker logs <container>` for application stdout/stderr; use `docker inspect` for the exit code and full config — exit code 137 usually means OOMKilled; check `docker ps -a` for an exit/restart loop pattern; exec into a related running container or attach a debug sidecar if the failing one can't stay up; and check resource limits, health check config, environment variables, and mounted volumes for misconfiguration.

61. Explain Kubernetes architecture.

Control plane: the API Server (front-end for all REST operations, the only component talking directly to etcd), etcd (distributed key-value store holding all cluster state), the Scheduler (assigns Pods to Nodes based on resources/constraints), and the Controller Manager (runs reconciliation loops like the ReplicaSet and Node controllers). **Worker node:** kubelet (talks to the API server, manages Pod lifecycle locally), kube-proxy (manages network rules for Services), and the container runtime (containerd/CRI-O).

62. Difference between Pod, ReplicaSet, and Deployment.

A **Pod** is the smallest deployable unit — one or more tightly coupled containers sharing network and storage. A **ReplicaSet** ensures a specified number of identical Pod replicas are running at all times, recreating them on failure. A **Deployment** manages ReplicaSets on top of that, adding declarative rolling updates, rollback history, and revision tracking — in practice you almost always create Deployments rather than bare ReplicaSets or Pods.

63. Explain StatefulSet.

Used for stateful applications (databases, Kafka, Zookeeper) that need stable, unique network identities (predictable Pod names like `web-0`, `web-1`), stable persistent storage tied to each Pod's identity via PersistentVolumeClaims that survive rescheduling, and ordered, graceful deployment, scaling, and termination — unlike Deployments, where Pods are interchangeable and disposable.

64. Difference between DaemonSet and Deployment.

A **DemonSet** ensures exactly one copy of a Pod runs on every (or a selected subset of) node automatically, used for node-level agents like log collectors, monitoring agents, or CNI plugins. A **Deployment** manages a desired number of replicas placed wherever the scheduler decides, independent of node count, used for typical stateless application workloads.

65. Explain Kubernetes Services.

A Service is a stable abstraction giving a consistent virtual IP/DNS name for accessing a dynamic, changing set of Pods (selected via label selectors), regardless of Pod restarts or rescheduling. It load-balances traffic across the currently healthy matching Pods using kube-proxy rules (iptables or IPVS).

66. Difference between ClusterIP, NodePort, and LoadBalancer.

ClusterIP (default) is only reachable inside the cluster, used for internal service-to-service communication. **NodePort** opens a static port (30000-32767) on every node, exposing the service externally via `<NodeIP>:<NodePort>` — mainly used for dev/testing or as a building block. **LoadBalancer** provisions an external cloud load balancer (e.g. an AWS NLB/ALB via the AWS Load Balancer Controller) that routes to the Service — the standard way to expose a Service externally in managed cloud clusters.

67. What is Ingress?

Ingress is an API object that manages external HTTP/HTTPS access to Services inside the cluster, providing host- and path-based routing, TLS termination, and a single entry point. It's handled by an Ingress Controller (e.g. NGINX, AWS Load Balancer Controller) that watches Ingress resources and configures the actual load balancer/proxy accordingly — more flexible and cost-effective than provisioning a separate LoadBalancer Service per app.

68. Explain ConfigMap and Secret.

Both decouple configuration from container images and can be mounted as volumes or injected as environment variables. **ConfigMap** stores non-sensitive plain-text key-value config (feature flags, app settings). **Secret** is meant for sensitive data (passwords, tokens, certs) — stored base64-encoded by default, which is *not encryption*, so production setups enable etcd encryption-at-rest or use a tool like Sealed Secrets / External Secrets Operator for real protection.

69. How does Kubernetes Service Discovery work?

Kubernetes runs an internal DNS service, CoreDNS, that automatically creates DNS records for every Service (e.g. `my-svc.my-namespace.svc.cluster.local`), letting Pods reach other Services by name instead of IP. Environment variables for Services are also injected into Pods at creation time as a legacy discovery mechanism, but DNS is the standard, recommended approach.

70. How do rolling updates work in Kubernetes?

When a Deployment's Pod template changes, the Deployment controller creates a new ReplicaSet and gradually scales it up while scaling the old ReplicaSet down, respecting `maxSurge` (extra Pods allowed above the desired count during rollout) and `maxUnavailable` (how many can be down at once) — achieving zero-downtime as long as readiness probes correctly gate traffic to only fully-ready new Pods. `kubectl rollout undo` reverts to the previous ReplicaSet if something goes wrong.

71. Explain a complete CI/CD pipeline.

Typically: a code commit/PR triggers the pipeline → **CI stage** (checkout, install dependencies, lint, run unit tests, build the artifact/Docker image, run static/security scans, push the artifact to a registry) → **CD stage** (deploy to a dev/staging environment, run integration/smoke tests, an automated or manual approval gate, deploy to production using a strategy like blue-green/canary/rolling, post-deploy verification, and automatic rollback if checks fail).

72. Difference between CI and CD.

Continuous Integration is the practice of frequently merging code changes into a shared branch, with automated builds and tests on every change to catch integration issues early. **Continuous Delivery/Deployment** extends this by automatically preparing (Delivery) or directly releasing (Deployment) every passing build to a production-like or production environment — Delivery keeps a manual approval gate before production, Deployment is fully automatic.

73. Explain Blue-Green Deployment.

Two identical, fully provisioned environments run in parallel: "blue" (current live) and "green" (new version). Traffic is switched from blue to green all at once — via DNS or a load balancer target group swap — once green passes validation. It enables near-instant rollback (switch back to blue) and zero-downtime releases, at the cost of running double the infrastructure during the transition.

74. Explain Canary Deployment.

A small percentage of production traffic (e.g. 5%) is routed to the new version while the majority continues hitting the old version; metrics and error rates are monitored, and traffic is gradually shifted to the new version if it's healthy, or instantly rolled back if not. This reduces the blast radius of a bad release compared to deploying to 100% of users at once.

75. Explain Rolling Deployment.

New version instances are deployed incrementally, replacing old instances a few at a time (controlled by batch size or `maxUnavailable`), until all instances run the new version. No duplicate full environment is needed, but rollback is slower since it requires another rolling update back to the old version, and both versions briefly coexist serving live traffic.

76. How do you implement rollback?

Keep every deployed artifact/image versioned and immutable so a known-good previous version can be redeployed quickly; use tooling with built-in rollback (`kubectl rollout undo`, CodeDeploy automatic rollback on alarm, ASG instance refresh rollback); automate rollback triggers tied to CloudWatch alarms/health checks rather than relying on manual action mid-incident; and design database migrations to be backward-compatible so rollback isn't blocked by schema changes.

77. How do you secure a CI/CD pipeline?

Use short-lived OIDC-based credentials instead of long-lived secrets in pipeline configs (e.g. GitHub Actions OIDC assuming an AWS role); scan dependencies and container images for vulnerabilities (SCA/SAST/DAST); enforce least-privilege IAM roles per pipeline stage; require code review and branch protection before merge; sign and verify build artifacts; restrict who can modify pipeline definitions; and audit pipeline access/changes via logging.

78. Explain artifact repositories.

Centralized storage for build outputs — JARs, Docker images, npm/pip packages, Terraform modules — with versioning, metadata, and access control. Examples include Amazon ECR/CodeArtifact, JFrog Artifactory, Nexus, and Docker Hub. They decouple build from deploy (build once, deploy the exact same immutable artifact to every environment), enable rollback to a specific version, and act as a single source of truth for what's actually running.

79. How do you manage environment-specific configurations?

Externalize config from code using environment variables, Parameter Store/Secrets Manager, or per-environment config files/values — separate Terraform tfvars, Helm values files, or Kustomize overlays. Never bake environment-specific values into the build artifact itself, so the exact same artifact can be promoted across dev → staging → prod, with only the surrounding configuration changing.

80. What causes deployment failures?

Common causes: configuration drift between environments, missing or incorrect environment variables/secrets, failed health checks due to slow app startup, insufficient IAM permissions for new resources, database migration failures or incompatibility with the still-running old version, resource limits set too low causing OOMKilled, network/Security Group misconfiguration blocking new resources, and untested edge cases that only surface under real production traffic or data volume.

81. Explain Terraform workflow.

Write configuration (.tf files) → `terraform init` (downloads providers/modules, configures the backend) → `terraform plan` (a dry run showing exactly what will change) → `terraform apply` (creates/updates/destroys resources to match the config, updating state) → ongoing iteration as the configuration evolves → `terraform destroy` when resources are no longer needed.

82. Difference between terraform init, plan, and apply.

`init` initializes the working directory — downloads required provider plugins and modules, and configures the backend for state storage. `plan` computes an execution plan by comparing current state with desired configuration, showing exactly what would be added, changed, or destroyed without making any real change. `apply` executes that plan, actually creating/modifying/destroying real infrastructure and updating the state file to reflect the new reality.

83. Explain Terraform State.

A JSON file (`terraform.tfstate`) that maps your configuration's resources to real-world object IDs, tracks metadata (dependencies, resource attributes), and is used to detect drift and compute diffs on every plan/apply. It should be stored remotely (e.g. S3 with DynamoDB for locking) rather than locally, since it's the source of truth Terraform relies on and often contains sensitive data.

84. What happens if the state file is deleted?

Terraform loses track of what it manages — the next plan would show it wanting to create everything from scratch (since it has no record of existing resources), risking duplicate resources or destructive changes if applied blindly. Recovery options: restore from a backup or a versioned S3 bucket, or manually rebuild state using `terraform import` for each existing resource — tedious but possible, which is exactly why remote state with versioning is critical.

85. Explain Terraform Modules.

Reusable, self-contained packages of Terraform configuration (input variables, resources, outputs) that can be called multiple times with different parameters — promoting DRY infrastructure code, consistency across environments, and encapsulation of complexity (e.g. a "vpc" module that creates subnets, route tables, and an IGW behind a simple interface). Modules can be local, from the Terraform Registry, or a Git repo.

86. Difference between count and for_each.

`count` creates N copies of a resource indexed by integer (0, 1, 2...) — simple but fragile, since removing an item from the middle of a list shifts all subsequent indices, causing Terraform to destroy and recreate resources unexpectedly. `for_each` iterates over a map or set of strings, creating one resource per key — each instance is tracked by its key rather than position, so removing one item only affects that specific resource, making it the safer choice for managing collections.

87. What is Terraform Drift?

Drift occurs when actual infrastructure differs from what's recorded in the state file — e.g. someone manually changed a resource in the console, or an external process modified it outside Terraform. `terraform plan` detects drift by refreshing state and comparing to the real resource; resolving it means either updating the configuration to match reality, or applying to force resources back to the declared configuration, ideally combined with stricter IAM policies to prevent manual changes going forward.

88. Explain Remote Backend.

A Remote Backend configures where Terraform stores its state file instead of locally — commonly S3 (paired with DynamoDB for state locking to prevent concurrent applies corrupting state) or Terraform Cloud. It enables team collaboration through shared state and locking, better security since state isn't sitting on individual laptops, and often built-in versioning and encryption.

89. How do you import existing infrastructure?

Use `terraform import <resource_address> <resource_id>` to bring an existing, manually-created (or otherwise unmanaged) resource into Terraform's state, then write matching configuration so a subsequent plan shows no diff. Newer Terraform versions also support a declarative, plannable `import` block in configuration. Care must be taken to match the configuration exactly, or Terraform will try to "fix" the resource on the next apply.

90. How do you handle secrets in Terraform?

Avoid hardcoding secrets in .tf files or committing them to version control; mark variables `sensitive = true` to hide them from plan/apply console output (though they're still stored in state in plaintext unless the state backend itself is encrypted); pull secrets at runtime from Secrets Manager, SSM Parameter Store, or Vault via data sources; encrypt the remote state backend (e.g. S3 with KMS); and restrict who can read the state file via IAM.

91. Explain the Linux boot process.

BIOS/UEFI performs POST and locates the boot device → the bootloader (GRUB) loads the kernel and an initial ramdisk (initramfs) into memory → the kernel initializes hardware, mounts the root filesystem, and starts the first process (PID 1, typically `systemd`) → `systemd` reads its targets/units and starts services in the configured default target (e.g. `multi-user.target` or `graphical.target`), bringing the system to a usable state.

92. How do you troubleshoot high CPU usage?

Start with `top` / `htop` to identify which process(es) are consuming CPU and whether it's user or system time; `ps aux --sort=-%cpu` for a quick snapshot; use `pidstat` or `strace -p <pid>` to see what syscalls a process is making; check for runaway loops or zombie processes; review application logs for the underlying cause (retry storms, GC thrashing); and use `vmstat` / `sar` to tell a sustained trend apart from a transient spike.

93. How do you troubleshoot high memory usage?

Use `free -h` for overall memory/swap usage; `top` / `htop` sorted by memory (%MEM/RES), or `ps aux --sort=-%mem`, to find the offending process; check for memory leaks by monitoring RSS growth over time; review `/proc/<pid>/status` for a detailed breakdown; check `dmesg` / `journalctl` for OOM-killer log entries; and check cgroup memory limits if running inside containers.

94. Explain systemd.

`systemd` is the modern init system and service manager (PID 1) on most Linux distros, replacing SysV init. It manages "units" — services, sockets, mount points, timers, targets — with parallelized startup, dependency-based ordering, and on-demand socket activation, plus automatic restart on failure. Key commands: `systemctl start/stop/restart/status <service>` and `journalctl -u <service>` for logs.

95. Difference between hard links and soft links.

A **hard link** is an additional directory entry pointing to the same inode (same underlying data) as the original file — it can't span filesystems, can't link to directories, and the data isn't actually deleted until every hard link is removed. A **symbolic (soft) link** is a separate small file containing a path/pointer to the target — it can span filesystems and link to directories, but breaks (a "dangling link") if the target is moved or deleted.

96. How do you troubleshoot disk space issues?

`df -h` to see overall filesystem usage per mount point; `du -sh /* 2>/dev/null` (or scoped per directory) to drill into which directories are consuming space; check for large or unrotated log files in `/var/log`, deleted-but-open files still held by a running process (`ls -l | grep deleted`), Docker dangling images/volumes (`docker system df / prune`), and old kernel versions or core dumps piling up; `ncdu` is a handy interactive tool for visualizing usage.

97. Explain process states in Linux.

Running (R) — actively executing or ready to run on CPU. **Sleeping (S)** — waiting on an event/resource, interruptible. **Uninterruptible Sleep (D)** — waiting on I/O that can't be interrupted, often a sign of disk/storage trouble if persistent. **Stopped (T)** — suspended, usually via SIGSTOP or job control. **Zombie (Z)** — the process has terminated but its parent hasn't yet reaped its exit status from the process table.

98. How do you troubleshoot a service that won't start?

`systemctl status <service>` for the immediate error and recent log lines; `journalctl -u <service> -xe` for the full detailed log/error context; verify config file syntax (most services have a test flag, e.g. `nginx -t`); check file/directory permissions and ownership the service needs; verify the port it wants to bind isn't already in use (`ss -tulpn`); check that dependent services (database, network) are up; and confirm the binary/path referenced in the unit file actually exists.

99. Explain SSH authentication and key exchange.

SSH first performs a key exchange (e.g. Diffie-Hellman/ECDH) to establish a shared session key over an otherwise insecure channel, encrypting all subsequent traffic; the server's host key is checked against the client's `known_hosts` to prevent a man-in-the-middle attack. For authentication, with key-based auth the client proves possession of the private key by signing a challenge the server verifies against the public key in `~/.ssh/authorized_keys` — the private key itself is never transmitted, which is why key-based auth is more secure than password auth.

100. Describe a production incident you handled and how you resolved it.

Frame this with the STAR method using a real incident from your own experience, or this representative structure: **Situation** — an ALB started returning 502s for an ECS Fargate service during a traffic spike. **Task** — restore service quickly and prevent recurrence. **Action** — checked target group health checks, found tasks being OOMKilled in CloudWatch Container Insights, correlated the memory growth to a recent deploy, rolled the task definition back to the previous stable revision, and temporarily raised the memory limit/desired count as a stopgap. **Result** — service restored within ~15 minutes, root-caused the leak with the dev team, and added a memory-utilization CloudWatch alarm plus a pre-deploy load test to catch it earlier next time. Swap in your own specific incident and numbers for authenticity in an actual interview.